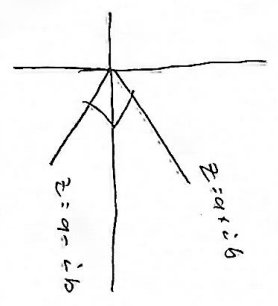


Recall: $z = a + ib$ (2d affine)

matrix $m \times n$ has $2mn$



Real $u, v \in \mathbb{R}^d$ $\langle u, v \rangle = \sum_{j=1}^d u_j v_j = u^T v$ symmetric + bilinear

Complex $u, v \in \mathbb{C}^d$ $\langle u, v \rangle = \sum_{j=1}^d u_j \overline{v_j} = u^T v^*$ $u^* = \overline{u}^T$

conj. sym $\langle u, v \rangle = \overline{\langle v, u \rangle}$
 sesq in inner conjugate then in
 (not, inner in sesq)

L layers
 14 effin heads
 d embedding dim
 V vocab size
 T max seq len
 $u, v \in \mathbb{C}^d$
 inner $\in \mathbb{C}^{mn}$
 $\tilde{z} = a + ib$ z real dot
 complex dot dim d
 exactly 2d real dot

Loss is not holomorphic $L: \mathbb{C}^n \rightarrow \mathbb{R}$ (not unimod), need Wirtinger calculus

$\partial_z = \frac{1}{2} (\partial_a - i \partial_b)$
 $\partial_{\bar{z}} = \frac{1}{2} (\partial_a + i \partial_b)$

Steepest ascent direction of L_{real} w.r.t to z is

$\partial_a L + i \partial_b L = 2 \partial_{\bar{z}} L$ $\text{PyTorch } g = \partial_a L + i \partial_b L$ as gradient
 with update $z \leftarrow z - \eta g$

$z' = (z - z_0) e^{i\theta} + z_0$ $\tilde{z} = z$ in $\mathbb{R}$ real case, so which is just calc
 $\uparrow$ grad point

Neer et al (2023) Wirtinger
 Kreutz-Delgado (2009) complex gradient

Token Embedding

word

stored in $W \in \mathbb{R}^{V \times 2d}$, reshaped to complex on forward pass

given $e \in \{0, \dots, V-1\}$

$$v_e = W[e, :] \in \mathbb{R}^{2d} \quad \left[\begin{array}{l} \text{use matrix complex embedding} \\ e_e = W[e, :] \in \mathbb{C}^d \end{array} \right]$$

$$e_e = \text{vector-as-complex}(v_e) \in \mathbb{C}^d$$

PyTorch implementation, no native $\mathbb{C}$ support

$$e_{e,j} = v_{e,2j} + i v_{e,2j+1} \quad \text{adjacent dimensions}$$

where consecutive pairs of real vals become real & imaginary

without it, embedding procedure requires some step at evaluation time
is also doing non-linearly
not present in brain

isomorphism $\mathbb{R}^{2d} \rightarrow \mathbb{C}^d$

$$\|v\|_{\mathbb{R}^{2d}}^2 = \sum_{j=0}^{2d-1} v_j^2 = \sum_{j=0}^{d-1} (v_{2j}^2 + v_{2j+1}^2) = \sum_{j=0}^{d-1} |e_j|^2 = \|e\|_{\mathbb{C}^d}^2$$

$$\text{So, } W \text{ has } 2Vd \text{ real params, } \partial_{e_b} L = \partial_{v_b} L$$

Plot matrix $\mathbb{C}^d$? Suppose we could via any ℓ_2 Gaussian columns. normalized

we use nn. Embedding $(V, 2d)$ (with default initialization)

uniform on $[-1, 1]/\sqrt{d}$ or normal (depends on version)

init differs [1. uses Pytorch default applied in $\mathbb{R}^{2d}$

no unit norm constants [2. No embedding onto unit sphere, so are unnormalized in $\mathbb{C}^d$ (no Heell=1)]

no 2^n constraint

no $d=2^n$ constraint
are embeddings
now directly to
n qubits.
we default to
 $d=256=2^8$

Position Encoding

we use fixed, non learned complex sinusoidal encodings

$$i \in \{0, \dots, T-1\} \quad \sin_{j \in \{0, \dots, d-1\}}$$

$$\text{angular freq } \omega_j = \frac{1}{10000^{2j/d}} \text{ and angle } \phi_{ij} = \omega_j \cdot i$$

$\omega_j \downarrow$ (from $\omega_0 = 1$ to $\omega_{d-1} = \frac{1}{10000}$)
 $\hookrightarrow$ low freq sinus oscillate very slowly
 $\hookrightarrow$ high freq.

$$PE_{ij} = \sin(\phi_{ij}) \text{ if } j \text{ is even}$$

$$\sin \theta \times \cos \theta = -i (\cos \theta \times i \sin \theta) = -i e^{i\theta}$$

every dim gets
 $\sin$ & $\cos$
 freq is even
 $\sin$ odd res
 $\hookrightarrow$ differentiable

$$= i e^{i \omega_j \cdot i} = -i e^{i \omega_j \cdot i}$$

$= -i e^{i \omega_j \cdot i}$
 $\hookrightarrow$ phase rotation or position

$\hookrightarrow$ then PE_{i+5}

$$|PE_{ij}| = 1$$

$\hookrightarrow$ pos emb contrib, higher phase, not magnitude

Same as freq, but every dim gets comp the opposite

- Non Learned?

$$\text{compit } W_P \in \mathbb{R}^{T \times d}$$

add $W_S \in \mathbb{R}^{2d \times d}$ segment, input to next layer

$$X_i^{(1)} = P_{x_i} \times W_P[i:] + W_S[W_P[i:], i] \in \mathbb{R}^d$$

$\hookrightarrow$ learned pos distributes system grad

$$\text{add } W_P \in \mathbb{C}^{T \times d}$$

$W_S \in \mathbb{C}^{2d \times d}$, $X_i \dots$ complex number addition

$$\text{mult } X_{2j}^{(1)} = (e^{i \omega_j \cdot j} \times \sin \omega_j \cdot j) \cdot e^{i \omega_j \cdot i}$$

each component rotated in $\mathbb{C}$ (Morgan's law gate in $\mathbb{Q}$)

learned pos updates not

$$PE_L = PE_L \cdot e^{-i \cdot i}$$